

Risk Assessment Prototype Testing Report

PHOENIX Project | AI/ML Integration and Cybersecurity Risk Validation

Purpose: This report summarises the testing completed for the PHOENIX risk assessment Python prototype. The prototype was created to show how AI/ML model output can be converted into a clear cybersecurity risk assessment response.

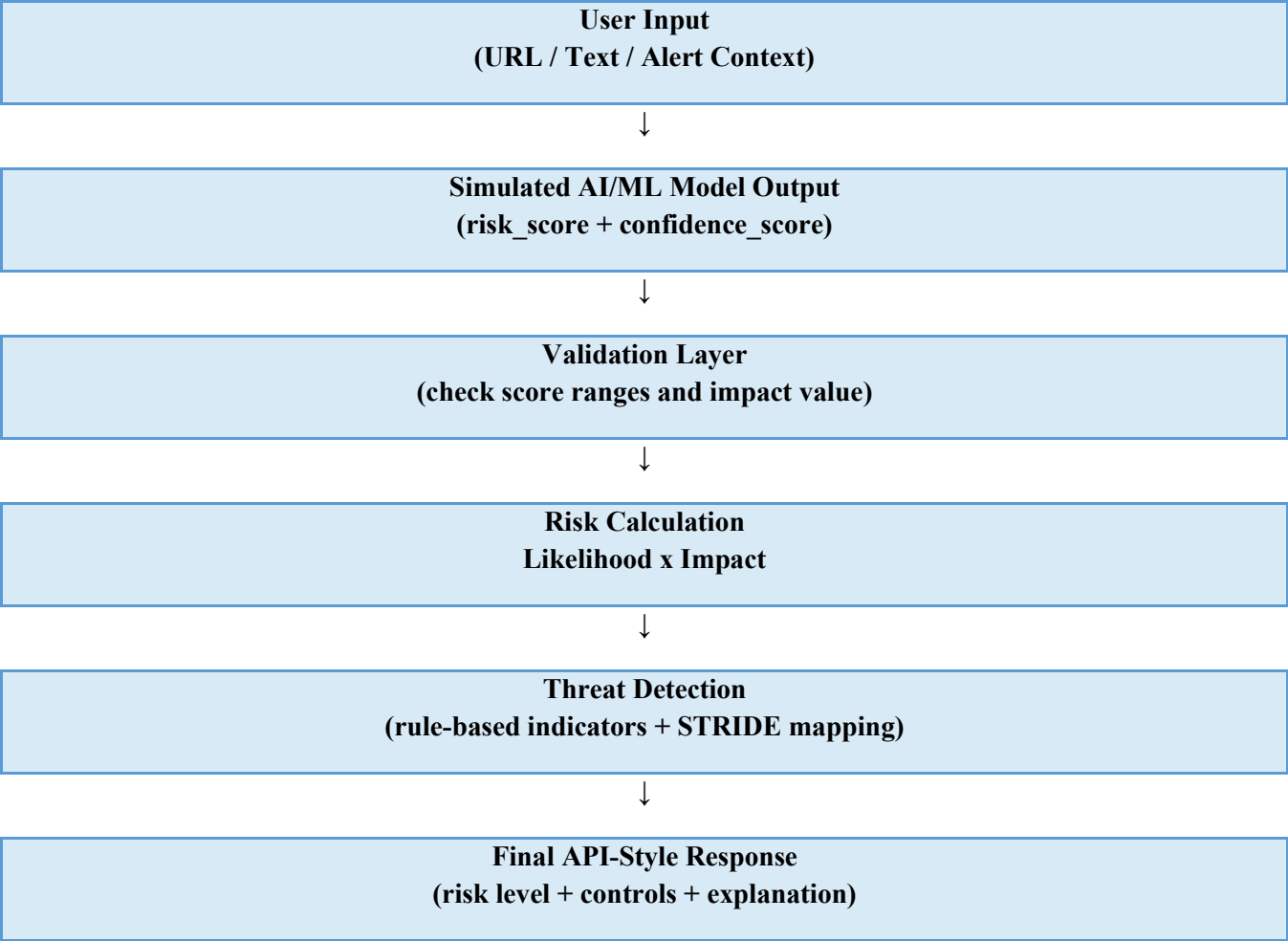
The test focuses on validating the risk logic, output structure, threat matching, STRIDE mapping, and recommended security controls. This is not the final deployed backend API; it is a working prototype that supports the API response schema and demonstrates technical feasibility.

Item	Summary
Task Area	CY019 - AI/ML Integration, risk scoring, output validation, and security mapping
Prototype Input	URL, text message, disaster context, alert level, source, and simulated AI model scores
Prototype Output	Legitimacy, likelihood, impact, final risk score, risk percentage, risk level, matched threats, controls, and explanation
Testing Goal	Confirm that the prototype returns a structured and meaningful API-style risk assessment response

Key idea: The Python prototype acts as a bridge between AI model output and the final cybersecurity response that the frontend/backend can use.

1. Prototype Workflow

The workflow below shows how the prototype processes an input and turns it into a final risk assessment result.



Explanation: The AI model provides the initial risk_score and confidence_score. The cybersecurity layer validates those values, calculates risk, checks threat indicators, and produces a structured response. This proves that the schema can be tested before the full backend integration is finalised.

2. Testing Method

The prototype was tested by running a sample high-risk disaster alert input through the Python file. The purpose was to confirm whether the code correctly calculates and returns the expected risk assessment output.

Test Element	Tested Value
URL	http://fake-bom-alert.com
Text	Emergency flood warning. Click this link now to verify your location.
Hazard Type	flood
Hazard Severity	high
Alert Level	emergency
Source	unknown
AI risk_score	0.88
AI confidence_score	0.91
Impact	5

Reason for this test case: This input was selected because it contains multiple high-risk indicators, including a fake-looking HTTP URL, urgent wording, emergency context, high hazard severity, and an unknown source.

Validation Check	Expected Result
risk_score between 0 and 1	Pass
confidence_score between 0 and 1	Pass
impact between 1 and 5	Pass
risk score calculation runs	Pass
API-style response is generated	Pass

3. Risk Calculation Results

The model risk score was converted into likelihood using the prototype mapping. Since the model risk score was 0.88, the likelihood value became 5, meaning highly likely.

Calculation Step	Result
Model risk score	0.88
Converted Likelihood	5
Impact	5
Final Risk Score	$5 \times 5 = 25$
Risk Percentage	$25 / 25 \times 100 = 100\%$
Risk Level	Critical
Legitimacy Result	Malicious

Main finding: The prototype successfully classified the input as malicious with a Critical risk level. This result matches the expected outcome because the input contains several strong indicators of phishing, fake alert activity, and misinformation risk.

Score Range	Risk Level
1-5	Low
6-10	Medium
11-17	High
18-25	Critical

4. Threat Detection and Control Mapping

The prototype detected multiple threat indicators from the tested input. These indicators were mapped to STRIDE categories and linked with recommended security controls.

Detected Threat	STRIDE Category	Why It Was Detected
Fake URL	Spoofing	The URL used HTTP and a suspicious/fake domain pattern.
Phishing Message	Spoofing	The message used urgent wording and asked the user to click a link.
Fake Disaster Alert	Tampering	The alert claimed emergency status but came from an unknown source.
Unverified Source	Spoofing	The submitted source was unknown.
Disaster Misinformation Risk	Tampering	High-severity disaster information came from an unverified source.
Recommended Control	Purpose	
URL Validation	Check whether the URL is trusted and not impersonating official sources.	
Domain Reputation Check	Identify suspicious or unsafe domains.	
Source Verification	Confirm whether the information comes from a trusted source.	
Content Filtering	Detect phishing-style wording and harmful message patterns.	
Input Validation	Validate URL, text, timestamps, hazard fields, and source values.	
Audit Logging	Record suspicious submissions for monitoring and accountability.	
Digital Signature Verification	Verify that disaster alerts were not forged or modified.	
JWT Authentication	Allow only authenticated users or trusted systems to access the API.	
Rate Limiting	Prevent spam requests, brute-force attempts, API abuse, and denial-of-service activity.	

5. Testing Evidence and Conclusion

The Python prototype was executed successfully in Visual Studio Code. The terminal returned a structured API-style response instead of an error. This confirms that the prototype logic is running correctly for the selected high-risk test case.

Important output evidence:

Output Field	Returned Value
status	success
legitimacy	malicious
likelihood	5
impact	5
final risk score	25
risk percentage	100
risk_level	Critical
matched_threats	Fake URL, Phishing Message, Fake Disaster Alert, Unverified Source, Disaster Misinformation Risk

Conclusion: The testing confirms that the prototype can take a simulated AI/ML output and convert it into a complete cybersecurity risk assessment response. The result includes risk calculation, legitimacy classification, matched threats, STRIDE mapping, security controls, and a clear explanation. This provides strong evidence that the CY019 response schema is not only theoretical but also testable through a working Python prototype.

Next steps: Test additional safe and medium-risk examples, compare expected vs actual outputs, and align the response format with the backend/API team once integration requirements are confirmed.